UTS DATA STRUCTURE AND ALGORITHM · SEMESTER 5 · 2026/2027

Cakrawala University

Smart Delivery & Order Optimization System

Kelompok 1

Dosen Pengampu: Lawrence Adi Supriyono, S.Kom., M.T.

Cakrawala University

Kelompok 1

Team	Responsibility
Atika	Lead Coordinator & Report
Husnu	Max-Heap
Eka	Priority Queue
Febri Anggoro	Hash Table (Chaining)
Rigan	Trie Autocomplete
Rendi	AVL Tree Ranking
M Febri	Red-Black Tree
Febri Andri	B-Tree Indexing
Daffa	Dynamic Programming

Cakrawala University

BUSINESS BACKGROUND

Food delivery harus memutuskan cepat, ribuan kali sehari

Setiap order harus diprioritaskan, dicari, dicocokkan dengan restoran, dan diantar lewat rute yang efisien.

[Konteks industri food delivery di Indonesia, 1–2 angka bersumber dari Proposal Bagian 3.1]

Pendekatan list biasa melambat secara linear saat data bertambah. Karena itu setiap kebutuhan dipetakan ke struktur data yang tepat.

100.000
order aktif per hari

10.000
restoran terdaftar

150.000
item menu

5–15
titik antar per kurir per batch

Asumsi desain sistem, bukan data industri

Cakrawala University

PROBLEM STATEMENT

Lima masalah yang harus dijawab rancangan kami

1	Order Urgent selalu diproses sebelum High dan Normal	Heap + Priority Queue
2	Order ditemukan dari order_id dalam rata-rata O(1)	Hash Table
3	Saran restoran muncul dari potongan kata yang diketik	Trie
4	Peringkat tetap terurut walau rating terus berubah	AVL Tree
5	Rute kurir tercepat tanpa mencoba semua n! urutan	Dynamic Programming

Cakrawala University

OBJECTIVES & FUNCTIONAL REQUIREMENTS

Tujuan dan fitur yang akan dibangun di UAS

Tujuan

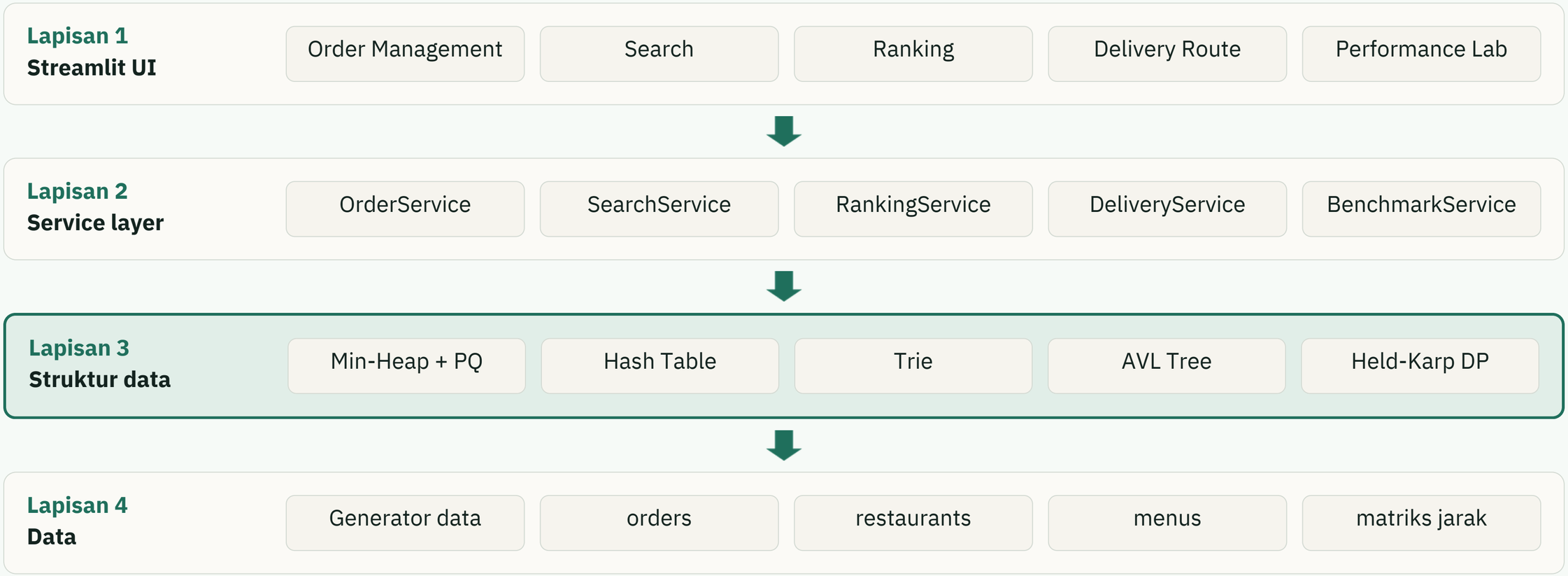
1. Priority Queue berbasis Min-Heap
2. Hash Table dengan Separate Chaining
3. Trie untuk prefix search
4. AVL Tree dengan 4 jenis rotasi
5. DP Held-Karp vs brute force
6. Rencana testing dan eksperimen

Modul	Fitur utama	ID
Order Management	Tambah, batalkan, ubah status order	FR-01–02
Priority	Lihat dan proses order paling urgent	FR-03–06
Search	Cari order, autocomplete restoran/menu	FR-07–10
Ranking	Top-k, filter rating, update rating	FR-11–14
Optimization	Rute optimal vs brute force	FR-15–16
Performance Lab	Eksperimen 1.000–100.000 data	FR-17

Cakrawala University

SYSTEM ARCHITECTURE

Empat lapisan, dari input pengguna sampai output



Mapping Fitur Bisnis ke Struktur Data

Fitur Bisnis	Struktur Data	Alasan Utama	Operasi Inti
Priority Order Management	Max-Heap + PQ	Order skor tertinggi selalu di root	Insert/Extract O(log N), Peek O(1)
Fast Order Search	Hash Table (chaining)	Lookup by Order ID tanpa scan	Search rata-rata O(1)
Restaurant/Menu Search	Trie	Satu jalur per prefix, cocok autocomplete	Prefix search O(L + K)
Restaurant Ranking	AVL Tree	Selalu seimbang, in-order = urutan ranking	Insert/Search O(log N)
Delivery Optimization	DP Bitmask	Sub-masalah rute berulang → memoization	$O(n^2 \cdot 2^n)$ vs O(n!)
Pembanding / Indexing	Red-Black & B-Tree	Dasar keputusan memilih AVL; index disk	O(log N) / O(log_t N)

N = jumlah order/restoran · L = panjang prefix · K = jumlah hasil · n = jumlah titik antar · t = minimum degree B-Tree

DATA STRUCTURE MAPPING

Setiap fitur dipetakan ke satu struktur data

Fitur bisnis	Struktur data	Kunci / isi	Operasi utama	Alternatif ditolak
Priority Order Management	Min-Heap + Priority Queue	(level, waktu, order_id)	insert, extract_min, peek	Sorted list, unsorted list
Fast Order Search	Hash Table (Separate Chaining)	order_id → Order	put, get, remove, rehash	Linear search, BST
Restaurant/Menu Search	Trie	huruf nama (huruf kecil)	insert, search, starts_with	Hash Table, sorted list
Restaurant Ranking	AVL Tree	(rating, restaurant_id)	insert, delete, top_k, range	BST biasa, Red-Black, B-Tree
Delivery Optimization	Dynamic Programming (Held-Karp)	dp[mask][j]	tabulation	Brute force, greedy

Prinsip pemilihan: struktur yang operasi utamanya paling sering dipakai fitur tersebut.

HEAP & PRIORITY QUEUE · 1/2

Order paling urgent selalu di puncak

Kunci = (level, waktu masuk)

1 = Urgent, 2 = High, 3 = Normal. Angka terkecil keluar duluan.

Level sama? FIFO

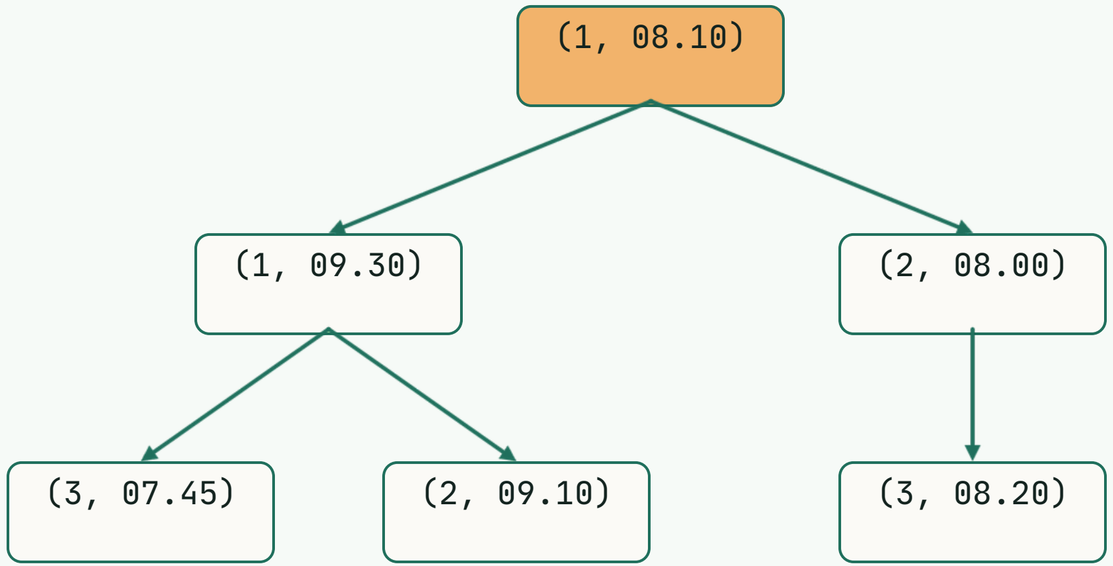
Yang masuk lebih dulu diproses lebih dulu.

Cepat walau data besar

Insert dan extract O(log n): 1 juta order hanya sekitar 20 tingkat.

Keterbatasan

Mencari order tertentu O(n), jadi pencarian diserahkan ke Hash Table.



Disimpan sebagai array, indeks 0 sampai 5:

(1,08.10)	(1,09.30)	(2,08.00)	(3,07.45)	(2,09.10)	(3,08.20)

Cakrawala University

Pseudocode inti: insert dan extract_min

```
FUNCTION insert(heap, order)
    APPEND (order.level, order.created_at, order.id) TO heap
    sift_up(heap, LENGTH(heap) - 1) // naik selama lebih kecil dari parent
FUNCTION extract_min(heap)
    IF heap kosong THEN RETURN NONE
    root ← heap[0]
    heap[0] ← elemen terakhir; hapus elemen terakhir
    sift_down(heap, 0) // turun ke anak terkecil
    RETURN root
```

insert	extract_min	peek
O(log n)	O(log n)	O(1)

Order ditemukan langsung dari nomornya

Kunci = order_id

Rolling hash polinomial menghasilkan indeks bucket; ukuran tabel bilangan prima.

Collision ditangani chaining

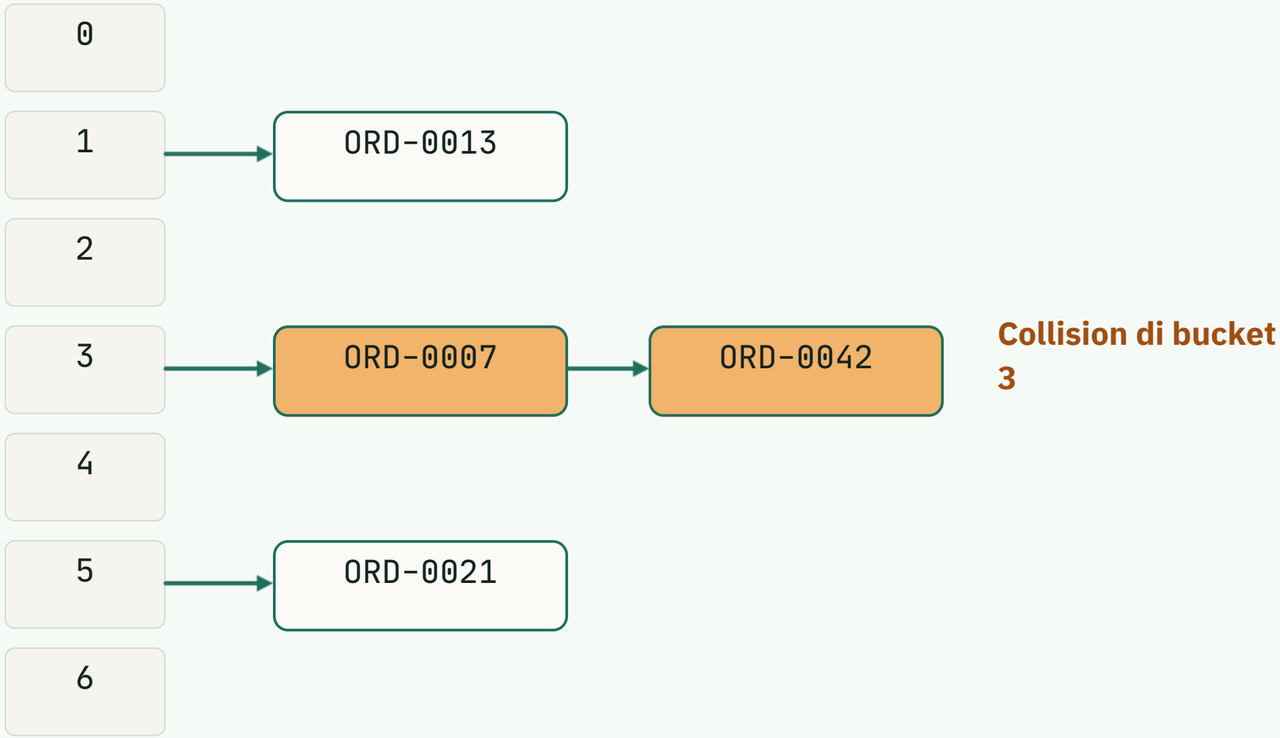
Dua kunci di bucket yang sama disimpan berderet dalam satu chain.

Kenapa bukan open addressing

Order sering dibatalkan atau selesai; menghapus di open addressing butuh tombstone.

Tetap cepat saat membesar

Load factor > 0,75 → tabel diperbesar 2× (rehash). Rata-rata O(1), terburuk O(n).



Kapasitas 7 (prima) · 4 order · load factor 4/7 ≈ 0,57

Cakrawala University

Pseudocode inti: put dan get

```
FUNCTION put(table, key, value)
    idx ← hash_function(key) // rolling hash mod kapasitas prima
    FOR (k, v) IN table[idx] DO
        IF k = key THEN ganti v; RETURN
    APPEND (key, value) TO table[idx]
    IF size / kapasitas > 0.75 THEN rehash(table)
FUNCTION get(table, key)
    FOR (k, v) IN table[hash_function(key)] DO
        IF k = key THEN RETURN v
    RETURN NONE
```

put / get / remove	kasus terburuk	rehash (jarang)
O(1) rata-rata	O(n)	O(n)

Saran restoran muncul saat pengguna mengetik

Ketik “ay” = 2 langkah

Setelah sampai di node y, semua kata di bawahnya menjadi saran.

Waktu O(m + k)

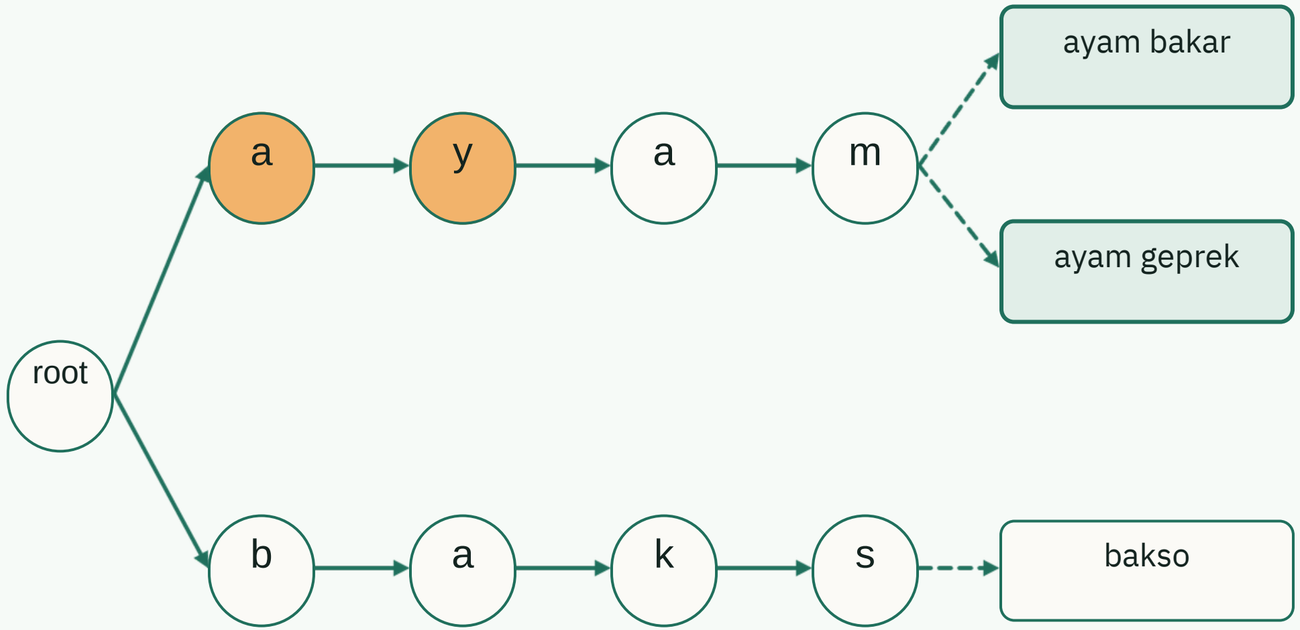
m = panjang ketikan, k = jumlah saran. Tidak bergantung jumlah restoran.

Kenapa bukan Hash Table

Hash Table hanya cocok untuk kata utuh, tidak bisa mencari awalan.

Keterbatasan

Boros memori karena setiap huruf menjadi node; hasil dibatasi 10 saran.



Oranye = jalur awalan “ay” · garis putus-putus = sisa huruf diringkas

Pseudocode inti: starts_with

```
FUNCTION starts_with(trie, prefix, limit)
    node ← trie.root
    FOR c IN lowercase(prefix) DO
        IF c NOT IN node.children THEN RETURN []
        node ← node.children[c]
    hasil ← []
    collect_words(node, prefix, hasil, limit) // DFS urut alfabet
    RETURN hasil
```

insert	search	starts_with
O(m)	O(m)	O(m + k)

Ranking tetap seimbang lewat rotasi

Kunci = (rating, restaurant_id)

ID memisahkan restoran yang rating-nya sama.

Selalu seimbang

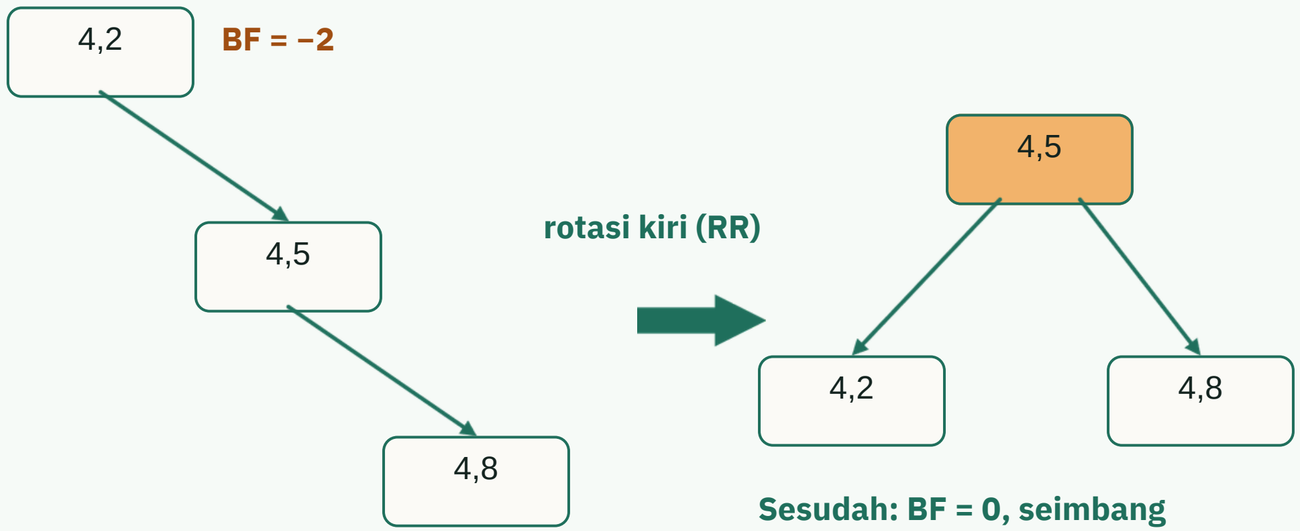
Selisih tinggi kiri dan kanan maksimal 1, sehingga operasi tetap O(log n).

Empat kasus rotasi

LL → kanan · RR → kiri · LR → kiri lalu kanan · RL → kanan lalu kiri

Kenapa bukan Red-Black / B-Tree

AVL lebih ketat seimbang sehingga pencarian lebih cepat; ranking lebih sering dibaca.



Sebelum: insert 4,2 → 4,5 → 4,8

Cakrawala University

Pseudocode inti: insert dengan empat kasus rotasi

```
FUNCTION insert(node, key)
    sisipkan seperti BST biasa, lalu perbarui tinggi node
    bf ← height(node.left) - height(node.right)
    IF bf > 1 AND key < node.left.key THEN RETURN rotate_right(node) // LL
    IF bf < -1 AND key > node.right.key THEN RETURN rotate_left(node) // RR
    IF bf > 1 THEN node.left ← rotate_left(node.left); RETURN rotate_right(node) // LR
    IF bf < -1 THEN node.right ← rotate_right(node.right); RETURN rotate_left(node) // RL
    RETURN node
```

insert / delete / search	top_k	satu rotasi
O(log n)	O(log n + k)	O(1)

Cakrawala University

DYNAMIC PROGRAMMING · 1/2

Rute kurir: urutan antar dengan waktu tersingkat

Masalah

Kurir membawa pesanan dari 1 restoran ke n customer. Cari urutan dengan total waktu minimum.

State

dp[mask][j] = waktu minimum mengunjungi semua titik dalam mask, berakhir di titik j.

Base case

dp[{R}][R] = 0, karena kurir mulai di restoran.

Jawaban akhir

Nilai terkecil dp[semua titik][j] untuk setiap j.

Contoh kecil: 1 restoran + 3 customer (menit)

	R	C1	C2	C3
R	0	4	6	9
C1	4	0	3	7
C2	6	3	0	2
C3	9	7	2	0

Rute terbaik: R → C1 → C2 → C3 = 9 menit

Hasil kecil dicatat, tidak dihitung ulang

```
dp[mask ∪ {k}][k] = min over j ( dp[mask][j] + dist[j][k] )
```

Brute force: coba semua urutan

$O(n \cdot n!)$

n = 10 → ±36 juta langkah

DP Held-Karp (tabulation)

$O(n^2 \cdot 2^n)$

n = 10 → ±100 ribu langkah

Cakrawala University

Big-O teoritis setiap struktur data

Struktur	Operasi	Rata-rata	Terburuk	Memori
Min-Heap / PQ	insert, extract_min	O(log n)	O(log n)	O(n)
Min-Heap / PQ	peek	O(1)	O(1)	O(n)
Min-Heap / PQ	build_heap	O(n)	O(n)	O(n)
Hash Table	put, get, remove	O(1)	O(n)	O(n)
Trie	insert, search	O(m)	O(m)	O(total huruf)
Trie	starts_with	O(m + k)	O(m + k)	O(total huruf)
AVL Tree	insert, delete, search	O(log n)	O(log n)	O(n)
AVL Tree	top_k	O(log n + k)	O(log n + k)	O(n)
DP Held-Karp	rute optimal	$O(n^2 \cdot 2^n)$	$O(n^2 \cdot 2^n)$	$O(n \cdot 2^n)$
Brute force	semua permutasi	$O(n \cdot n!)$	$O(n \cdot n!)$	O(n)

n = jumlah data · m = panjang kata atau prefix · k = jumlah hasil · pada DP, n = jumlah titik antar

RENCANA TESTING & PERFORMANCE EXPERIMENT

18 test case dan 5 eksperimen untuk UAS

18 test case

Modul	Test case	Fokus
Heap & PQ	TC-01–04	prioritas, FIFO, heap kosong
Hash Table	TC-05–08	collision, rehash, remove
Trie	TC-09–11	prefix ada/tidak, huruf besar
AVL Tree	TC-12–15	rotasi LL, RR, LR, RL
DP	TC-16–17	hasil = brute force
Integrasi	TC-18	order sampai rute

5 eksperimen performa

	Struktur data vs baseline
E1	Min-Heap vs sorted list
E2	Hash Table vs linear search
E3	Trie vs linear startswith
E4	AVL vs BST tanpa balancing
E5	Held-Karp vs brute force

1.000 / 10.000 / 50.000 / 100.000 data · waktu, jumlah operasi, memori · grafik di Streamlit

Cakrawala University

TIMELINE & PEMBAGIAN TUGAS

Rencana enam minggu menuju UAS

Minggu 1

Setup repo, generator data, service layer

Architect, QA

Minggu 2

Implementasi Heap, Hash, Trie, AVL + unit test

4 DS Engineer

Minggu 3

DP, brute force, integrasi modul

DP, Ketua

Minggu 4

Halaman Streamlit semua modul

Architect + semua

Minggu 5

Test plan dan eksperimen performa

QA + semua

Minggu 6

Laporan, slide, latihan presentasi

Ketua + semua

Kesimpulan: setiap fitur sudah punya struktur data, justifikasi, pseudocode, rencana uji, dan pemiliknya. Kami siap masuk tahap implementasi.

Cakrawala University

REFERENSI

Sumber teori rancangan

1. Adelson-Velsky, G. M., & Landis, E. M. (1962). An algorithm for the organization of information. Soviet Mathematics Doklady, 3, 1259–1263.
2. Bellman, R. (1957). Dynamic programming. Princeton University Press.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.
4. Fredkin, E. (1960). Trie memory. Communications of the ACM, 3(9), 490–499.
5. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data structures and algorithms in Python. Wiley.
6. Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. Journal of SIAM, 10(1), 196–210.
7. Knuth, D. E. (1998). The art of computer programming, Vol. 3: Sorting and searching (2nd ed.). Addison-Wesley.
8. Streamlit Inc. (n.d.). Streamlit documentation.